

Part 8: Balanced Binary Tree

And the sentinel value pattern

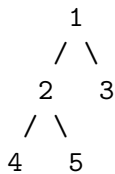
What balanced means

LeetCode 110 uses this definition:

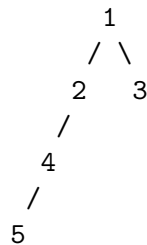
A binary tree is height balanced if, **for every node**, the heights of its left and right subtrees differ by at most 1.

The phrase “for every node” is the whole problem. It is not enough to check the root.

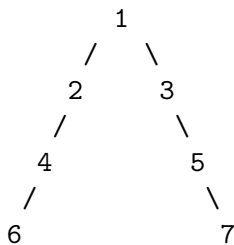
balanced



NOT balanced



In the right hand tree, look at the root: left height 3, right height 1. Difference 2. Broken. But you can also build trees where the root looks perfectly fine and a node deep inside is broken:



At the root: left height 3, right height 3, difference 0, looks fine. At node 2: left height 2, right height 0, difference 2. Not balanced. If you only check the root you will answer “yes” and be wrong.

Why do we care about this property at all? Because balance is exactly what guarantees height $O(\log N)$, which is what makes BST operations fast. AVL trees and red-black trees exist to maintain this invariant automatically.

Naive solution

Straight translation of the definition:

```
int height(TreeNode* root) {
    if (root == NULL) return 0;
    return 1 + max(height(root->left), height(root->right));
}

bool isBalancedNaive(TreeNode* root) {
    if (root == NULL) return true;

    int L = height(root->left);
    int R = height(root->right);
    if (abs(L - R) > 1) return false;           // check me

    return isBalancedNaive(root->left) &&      // check everyone below me
           isBalancedNaive(root->right);
}
```

This is correct. It is also the same trap as the diameter brute force: at every node we call `height`, which walks the whole subtree again.

Worst case (skewed tree):

```
node 1: height walks N nodes
node 2: height walks N-1 nodes
...
total O(N^2)
```

Balanced input gives $O(N \log N)$, which is why the naive version sometimes passes small tests and dies on the big ones.

The $O(N)$ solution: the sentinel trick

We want height and balance in the same pass. There are two clean ways.

Way 1: return -1 to mean “already broken”

The height function normally returns a value that is 0 or larger. So the value -1 is unused, and we can hijack it to carry a second message: **this subtree is not balanced, stop caring about its height.**

```
class Solution {
    // returns the height, or -1 if this subtree is not balanced
    int check(TreeNode* root) {
        if (root == NULL) return 0;

        int L = check(root->left);
        if (L == -1) return -1;           // left is broken, propagate

        int R = check(root->right);
```

```

        if (R == -1) return -1;           // right is broken, propagate

        if (abs(L - R) > 1) return -1;    // I am broken

        return 1 + max(L, R);           // fine, report my height
    }

public:
    bool isBalanced(TreeNode* root) {
        return check(root) != -1;
    }
};

```

Once any node reports -1, every ancestor immediately returns -1 without doing more work. The failure travels straight to the top.

Time $O(N)$ (each node visited once), **space $O(H)$** .

Way 2: return a small struct

If the sentinel feels like a hack, or if -1 could be a legal value in some other problem, return both facts explicitly:

```

struct Info {
    bool balanced;
    int height;
};

Info check(TreeNode* root) {
    if (root == NULL) return {true, 0};

    Info L = check(root->left);
    Info R = check(root->right);

    Info me;
    me.height = 1 + max(L.height, R.height);
    me.balanced = L.balanced && R.balanced && abs(L.height - R.height) <= 1;
    return me;
}

bool isBalanced(TreeNode* root) {
    return check(root).balanced;
}

```

Same complexity. More typing, zero cleverness required to read it. Both are perfectly acceptable in an interview. Say out loud which one you are doing and why.

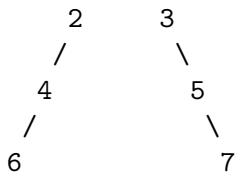
Dry run of the sentinel version

Tree:

```

    1
   / \

```



```

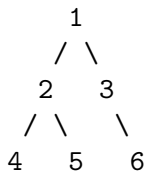
check(6): L=0 R=0          |0-0|=0 ok      return 1
check(4): L=1 R=0          |1-0|=1 ok      return 2
check(2): L=2 R=0          |2-0|=2 BAD     return -1
check(1): L = -1           propagate      return -1

```

```
isBalanced = false
```

Notice that once node 2 returned -1, the entire right subtree of the root was never even visited. The `if (L == -1) return -1;` line before computing R gives you that early exit for free. Put it there.

Now our familiar tree:



```

check(4) = 1
check(5) = 1
check(2): |1-1| = 0 ok      return 2
check(6) = 1
check(3): L=0 R=1, |0-1|=1 ok return 2
check(1): |2-2| = 0 ok      return 3

```

```
isBalanced = true
```

The pattern, stated generally

This is bigger than one problem. The pattern is:

When a recursive function must return a value **and** report a failure, and there is an impossible value in the normal range, use that impossible value as the failure signal and propagate it upward immediately.

Examples of the same idea:

- Balanced tree: height, or -1 for “not balanced”.
- Largest BST subtree: (size, min, max), or a flag saying “not a BST”.
- Checking whether all values are within a range: return the range, or an impossible range.
- Many graph problems: return a distance, or -1 for unreachable.

The one rule: **make sure the sentinel cannot occur legitimately**. If node values can be negative and you are returning sums, -1 is a terrible sentinel. Then use the struct.

Complexity summary

Version	Time	Space	Comment
Naive, height at every node	$O(N^2)$ worst case	$O(H)$	correct but slow
Sentinel -1	$O(N)$	$O(H)$	shortest accepted solution
Struct	$O(N)$	$O(H)$	clearest to explain

Common mistakes

- **Checking only the root.** The definition says every node.
- **Computing height inside `isBalanced` at every node.** The $O(N^2)$ trap.
- **Using -1 when -1 is a legal answer.** Then you cannot tell failure from a real value.
- **Forgetting to propagate the failure.** If a child returns -1 and you go on to compute $1 + \max(L, R)$, you produce a height of 0 or 1 and the ancestors happily report “balanced”.
- **Using `abs(L - R) >= 1`.** The condition is a difference of at most 1, so the failure test is > 1 .
- **Confusing this with “complete” or “perfect”.** Balanced is a much weaker condition. Our sample tree is balanced but neither complete nor perfect in the strict sense.

Check yourself

1. Why is checking only the root insufficient? Draw a tree that fools it.
2. What makes -1 a safe sentinel for a height function?
3. Where in the sentinel code does the early exit happen, and how much work does it save?
4. Rewrite the sentinel version’s meaning in one sentence: “`check(x)` returns ...”.
5. If node values could be negative and you were returning subtree sums, why would -1 be a bad sentinel, and what would you do instead?
6. Is every balanced tree complete? Is every complete tree balanced?